

Aula 10 - Integração Contínua (CI/CD) com Automação de Qualidade

Duração: 2 horas

Objetivos:

- Configurar pipelines CI/CD com GitHub Actions e Jenkins
- Implementar estágios automatizados de build, teste e deploy
- Integrar ferramentas de qualidade no pipeline

Parte Teórica (40 min)

1. CI/CD: Conceitos Fundamentais

Estágio	Ferramentas	Objetivo
Build	Maven, Gradle	Compilar código e resolver dependências
Test	JUnit, TestContainers	Validar funcionalidades automaticamente
Quality Gate	SonarQube, Checkstyle	Bloquear código com baixa qualidade
Deploy	Docker, Kubernetes	Publicar em ambientes de homologação/produção

2. GitHub Actions vs Jenkins

Critério	GitHub Actions	Jenkins
Configuração	YAML no repositório	UI ou Jenkinsfile (Groovy)
Escalabilidade	Runners pré-configurados	Requer infraestrutura própria
Integração	Nativo com GitHub	Multiplataforma

3. Pipeline de Qualidade Ideal

```
graph LR
  A[Commit] --> B[Build]
  B --> C[Testes Unitários]
  C --> D[Análise Estática]
  D --> E{Passou?}
  E -->|Sim| F[Deploy Staging]
  E -->|Não| G[Notificação de Falha]
```

Parte Prática (1h20min)

Atividade 1: Pipeline CI com GitHub Actions

1. Criar `.github/workflows/ci.yml`:

```
yaml

name: Java CI Pipeline

on: [push, pull_request]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Build with Maven
        run: mvn clean package

      - name: Run Unit Tests
        run: mvn test

      - name: Code Quality Check
        run: mvn checkstyle:check
        continue-on-error: false
```

2. Adicionar Quality Gate com SonarQube:

```
yaml
```

```
- name: SonarQube Scan
uses: SonarSource/sonarqube-scan-action@master
env:
  SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
  SONAR_HOST_URL: ${ secrets.SONAR_HOST_URL }
```

Exercício:

- Modifique o pipeline para gerar relatório JaCoCo
- Configure um approval manual antes do deploy

Atividade 2: Pipeline CD com Jenkins

1. Criar Jenkinsfile no projeto:

groovy

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }
        stage('Test') {
            steps {
                sh 'mvn test'
                junit 'target/surefire-reports/**/*.xml'
            }
        }
        stage('Deploy to Staging') {
            when {
                branch 'main'
            }
            steps {
                sh 'docker build -t myapp:${GIT_COMMIT} .'
                sh 'kubectl apply -f k8s/deployment.yaml'
            }
        }
    }
    post {
        failure {
            slackSend channel: '#alerts', message: "Build Failed: ${env.JOB_NAME}"
        }
    }
}
```

```
}  
}
```

2. Configurar webhook no GitHub:

- Settings → Webhooks → `http://jenkins-server/github-webhook/`

Exercício:

- Adicione um estágio para análise PMD
- Implemente rollback automático em caso de falha no deploy

Atividade 3: Automação de Qualidade

1. Bloquear PRs com falhas:

yaml

```
# Em ci.yml  
name: Quality Gate  
on: pull_request  
  
jobs:  
  sonarqube:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v3  
      - name: SonarQube Scan  
        uses: SonarSource/sonarqube-scan-action@master  
        with:  
          args: >  
            -Dsonar.qualitygate.wait=true
```

2. Configurar regras no SonarQube:

- Cobertura mínima: 80%
- Zero bugs críticos
- Debt técnico < 1h

Exercício:

- Simule um PR com baixa cobertura e observe a rejeição

Desafio Integrado (30 min)

Contexto:

1. Crie um pipeline que:
 - o Rode em paralelo:
 - Testes unitários
 - Testes de integração com TestContainers
 - o Só faça deploy se:
 - Todos os testes passarem
 - Cobertura > 85%
 - Não houver vulnerabilidades no OWASP Dependency-Check

Dicas:

yaml

```
jobs:
  unit-tests:
    # ...
  integration-tests:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: mvn verify -Pintegration-tests
```

Encerramento

- **Discussão:** Qual abordagem (GitHub Actions/Jenkins) é melhor para seu projeto?
- **Próxima aula:** Análise Estática e Qualidade de Código

Tarefa:

- Implementar um pipeline completo para seu projeto final
- Adicionar verificação de segurança com OWASP Dependency-Check

Recursos:

- [GitHub Actions Marketplace](#)
- [Jenkins Pipeline Syntax](#)

Prontos para automatizar seu fluxo de entrega? Dúvidas? 😊🚀